

FACULDADE DE CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D COIMBRA

Meta 2

Desenvolvimento de sistemas centrados em bases de dados

Projeto realizado por:

Brenda de Abreu Nº2019112462 (uc2019112462@student.uc.pt)

Inês Nisa Nº2022247346 (uc2022247346@student.uc.pt)

Miguel Calado Nº2022228235 (uc2022228235@student.uc.pt)

Turma PL6

INTRODUÇÃO

No âmbito da cadeira de Base de Dados foi-nos proposto a realização de um projeto em que proporcione o desenvolvimento de sistemas centrados em bases de dados.

Este projeto consiste no desenvolvimento de um Sistema de gestão hospitalar, suportado por um sistema de gerenciamento de banco de dados, desenvolvido em Python e utilizando o PostgreSQL.

No desenvolvimento da base de dados atendemos às funcionalidades pedidas e às restrições para garantir o armazenamento eficaz e o processamento e recuperação de informações.

MANUAL DE INSTALAÇÃO

Para realizar o presente trabalho utilizamos linguagem python bem como sql, nós escolhemos utilizar como ambiente de desenvolvimento o pycharm, como base de dados o PostgreSQL e como ferramenta de testes de API o postman. Dentro do PyCharm, instalamos os pacotes necessários, incluindo Flask e Psycopg2.

O FLASK permite criar endpoints para interagir com a base de dados e responder a requisições HTTP.

```
@app.route('/patient', methods=['POST'])
```

Em cada endpoint definimos a rota e o método HTTP correspondente (‘GET’, ‘POST’, ‘PUT’) tal como mostrado nesta linha, onde define a rota /patient e o método HTTP ‘POST’ . O psycopg2 é o que permite que a aplicação FLASK conecte-se à base de dados PostgreSQL, com ele conseguimos executar comandos SQL diretamente no código python. O Flask_JWT-Extended é uma extensão do FLASK que é usada para a autenticação baseada em tokens para que só os usuários com login possam aceder às rotas. Para além destas também instalamos a biblioteca Bcrypt que usamos para encriptar as senhas dos utilizadores serem assim armazenadas de forma segura no banco de dados.

Para obter os dados enviados na requisição HTTP como um objeto JSON usamos esta linha de código (ex: obter os dados enviados pelo utente para depois serem processados):

```
payload = flask.request.get_json()
```

A aplicação do postman permite enviar requisições HTTP para os endpoints e verificar as respostas, um exemplo da sua utilização:

```
{
    "nif": "123456789",
    "nome": "Nome do Enfermeiro",
    "telemovel": "987654",
    "email": "enfermeiro@example.com",
    "data_nasc": "1999-08-19",
    "genero": "Masculino",
    "password": "1234ola2"
}
```



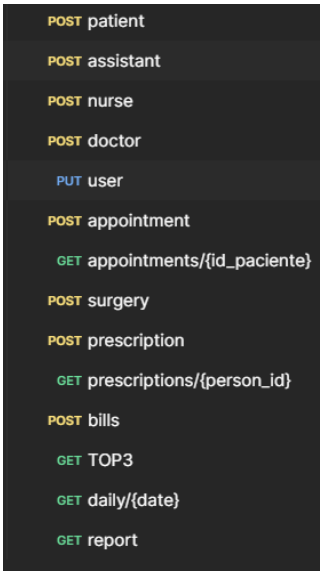
Para configurar a base de dados PostgreSQL inserimos as tabelas que tínhamos já provenientes da meta1 através do ONDA. Com estas tabelas configuradas podemos começar a aplicação, uma vez que podem já ser armazenadas as informações e assim conseguimos proceder com a integração com o banco de dados para permitir as operações desenvolvidas.

Para estabelecer ligação com a base de dados em cada endpoint temos de fazer isto:

```
conn = db_connection()
cur = conn.cursor()
```

O que estas duas linhas fazem é basicamente conectar à base de dados e após estabelecer ligação cria um cursor para permitir executar comandos SQL, ou seja, é meio que uma ponte entre o código e a base de dados.

Na aplicação Postman criámos vários requests para executar os endpoints que criámos no código, tal como mostramos a seguir:



Fizemos import dos requests para o ficheiro BD_projeto.postman_collection.json para ser possível testar todas as funcionalidades da aplicação.

Antes de tudo é necessário executar o código dos Triggers e dos Procedures dentro da base de dados, isto porque os triggers garantem que as ações que realizamos sejam executadas automaticamente em resposta a acontecimentos específicos conforme o que vai acontecendo. Assim, executar os Triggers e as Procedures antes garante a lógica da base de dados fazendo com que a aplicação funcione corretamente como esperado.

MANUAL DE UTILIZADOR

- **ENDPOINT**
  ADD_PATIENT/ADD_ENFERMEIRO/ADD_MEDICO/ADD_ASSISTANT

Começamos por extrair o payload JSON, iniciamos a variável 'require_fiels' com os campos necessários para adicionar um paciente e de seguida verificamos se não há nenhum campo em falta se houver retorna mensagem de erro. De seguida estabelecemos ligação com a base de dados e o payload é registado. Encriptámos a palavra-passe proveniente de modo que não seja acedida por outros, para isso importamos a package bcrypt. De seguida é chamada a procedure ('add_patient' ou 'add_enfermeiro' ou 'add_assistant' ou 'add_medico'), esta procedure tem como parâmetros de entrada aqueles que foram extraídos do postman e como parâmetros de saída tem o id do novo utilizador, se a operação foi 'success' ou 'error' e também a mensagem com o resultado. Depois iniciamos a procedure, damos locks na tabela de utilizador e na tabela da pessoa estamos a adicionar (paciente, médico, enfermeiro, assistente) no modo exclusivo para evitar concorrência. Como não pode haver nifs iguais verificamos se o nif já existe e se já existir manda 'error', se não existir continua e adiciona o novo utilizador na tabela 'utilizador' bem como na tabela correspondente associado ao id criado no utilizador. No final manda a mensagem de 'succes' e 'Paciente/Enfermeiro/Médico/Assistente adicionado com sucesso!'. Se houver alguma exceção não especificada manda a mensagem de 'error'. De seguida voltando ao ficheiro python é chamada 'cur.fetchone' de modo a ir buscar o resultado da chamada da procedure, se a resposta da procedure for 'success' e realiza o commit da transação no banco de dados caso contrário faz rollback. Por fim são tratadas as exceções e a conexão à base de dados é fechada.

- **LOGIN/AUTHENTICATE**

Na função 'def login()' começamos por extrair o payload JSON da requisição de seguida obtemos o nif do payload JSON bem como a palavra-passe. De seguida chamamos a função authenticate que recebe o nif e o password como parâmetros. Se as credenciais forem válidas regista uma mensagem a dizer que o usuário foi autenticado com sucesso de seguida cria um token de acesso JWT usando o nif como identidade, este depois vai ser usado para nas outras funções sabermos quem fez login, por fim retorna esse token. Se as credenciais forem inválidas regista a mensagem a dizer que falhou o login. O token vai ser recebido e vai ser posto no header.

A função 'authenticate' estabelece ligação com a base de dados e define uma consulta sql para ir buscar a password conforme o nif que foi enviado como parâmetro da função. O ‘cur.execute’ executa a consulta sql substituindo o ‘%s’ pelo nif e em seguida guarda o resultado da consulta na variável ‘result’. Em seguida verifica se retornou algo e recupera a senha armazenada do resultado. Como a senha foi encriptada vai verificar se corresponde e se coincidirem retorna ‘True’. No final vai capturar qualquer exceção que tenha ocorrido e fecha a conexão com a base de dados.

- **CREATE APPOINTMENT**

Começámos por guardar o nif do usuário autenticado a partir do token e guardamos na variável patient_nif e em seguida obtemos o id do paciente a partir do nif . Depois verificamos se o usuário é um paciente usando a função ‘is_patient’ e se não for damos return da resposta de erro. Se for paciente vamos extrair os dados do payload e verificamos se todos os campos obrigatórios foram inseridos, senão retornamos uma mensagem de erro. Definimos o nivel de isolamento da conexão como serializable e iniciamos uma transação e inserimos a consulta na base de dados usando ‘cur.execute’ e passando como valores a data, o id do paciente e do medico. Em seguida verificamos se existe algum id de enfermeiro e se houver associamos enfermeiros á consulta e depois retornamos uma resposta de sucesso

- **PROCESS PAYMENT**

Esta função só aceita requisições ‘POST’ e também que só usuários que tenham feito o login o podem fazer ‘jwt_required()’. Começamos por obter o nif a partir do token, depois estabelecemos ligação com a base de dados e criamos um cursor para executar os comandos SQL. Depois executamos uma consulta SQL em que vai buscar o id da pessoa que fez login através do nif e armazena em ‘user_id’, de seguida faz outro comando para verificar se o utilizador é um paciente antes de fazer as outras operações. Continuamente depois vamos extrair os dados do pagamento do payload JSON e verifica se a conta pertence ao utilizador que fez login, se for do paciente vai atualizar o ‘valor_pago’ e vai buscar o valor do ‘preço’ e de seguida vai ver se o valor pago é maior ou igual ao ‘preço_total’ e se for manda mensagem de pagamento concluído.

- **GET APPOINTMENT**

Começamos por estabelecer ligação à base de dados e criar um cursor. De seguida verificamos se o id é válido e se for chamamos o procedimento 'get_patient_appointments' em q o id é um dos parâmetros e extrai o resultado desse procedimento em 'results'. O procedimento sql o que faz é ir procurar o id do paciente na tabela utente e executa uma consulta para ir buscar os dados do paciente e atribui esses valores aos parâmetros de saída, se não existir um registro na tabela 'útente' manda mensagem de erro. Voltando ao python ele verifica se a mensagem foi de sucess e se for extrai os dados do agendamento da consulta e cria uma resposta JSON, se não for sucess manda mensagem de erro.

- **MARCAR_CIRURGIA**

Começamos por extrair o nif do usuário autenticado e abrir a conexão com a base de dados e depois vamos obter o id a partir do nif extraído e guardar na variável user_id. Verificamos se o usuário autenticado é médico ou paciente e se for damos return a uma mensagem de erro pois apenas os assistentes podem marcar cirurgias, extraímos os dados do payload e vamos verificar se todos os campos obrigatórios foram inseridos e verificamos também se a lista de enfermeiros está vazia e se o número do quarto já está ocupado ou não e se uma ou outra não se verificarem vamos dar return a uma mensagem de erro. Depois extraímos o doente primário da lista de enfermeiro e verificamos se existe na base de dados. Se já houver um internamento ele apenas atualiza os dados senão cria um internamento novo e em seguida inserimos a cirurgia na base de dados e os enfermeiros relacionamos e retornamos uma resposta de sucesso. No final vai capturar qualquer exceção que tenha ocorrido e fecha a conexão com a base de dados.

- **GET PRESCRIPTIONS**

Começamos por estabelecer ligação à base de dados e criamos um cursor, de seguida obtemos o nif do utilizador autenticado. Em seguida realizamos uma consulta para ir buscar o nif com base no id que veio da função, se o utilizador for encontrado verifica se é paciente e se for verifica se o id é diferente ao id que veio do cabeçalho da função e se for manda mensagem de erro, depois também verifica se o autenticado é médico e se for manda mensagem a dizer q n tem acesso. De seguida chama o procedimento 'get_prescriptions' que passa como parâmetro o id da pessoa e none para os parâmetros de saída, depois extrai os resultados para 'result' e vai verificar se recebeu 'sucess'

mandando assim a mensagem com as prescrições daquele paciente, caso contrário manda mensagem de erro.

O procedimento em sql o que faz é verificar se o usuario existe caso contrario manda mensagem de erro, caso exista vai buscar o nif e aramazena e executa uma consulta para ir buscar todas as receitas do utilizador em questão, armazenado todos esses objetos JSON em um array JSON.

- **ADD PRESCRIPTIONS**

Começamos por obter o nif do usuário autenticado e guardar na variável doctor_nif e abrir a conexão com a base de dados. Depois verificamos se o usuário autenticado é um médico e se não for retornamos uma mensagem de erro.Em seguida vamos obter o id do usuário a partir do nif e extraímos os dados do payload e verificamos se todos os campos foram inseridos. fazemos um if para verificar se é uma consulta ou um internamento ou outra opção inválida, se for consulta ele vai inserir nas tabelas 'receita' o id do utilizador e a data e na tabela 'receita_consulta' o id da receita e o id da consulta e na tabela 'medicamento_dosagem' vamos inserir o nome do medicamento ou medicamentos e as suas dosagens e o id da receita, para o internamento vai ser semelhante apenas inserimos na tabela 'internamento_receita' o id da receita e o id do internamento. Depois de inserimos vamos retornar uma mensagem de sucesso e vamos capturar qualquer exceção que tenha ocorrido e fechar a conexão com a base de dados.

- **ENDPOINT TOP_3**

Começamos por inicializar um dicionário que vai guardar informações sobre o top 3(preço total e id do utente), depois verificamos se ele era assistente e se não for retorna uma mensagem de erro. De seguida usamos o 'cur.execute' para ir buscar o id de todos os utilizadores e guardamos na variável 'todosUtentes_id', se essa variável estiver vazia retornamos uma mensagem de erro.Vamos fazer um ciclo for para iterar sibre cada id de utente em eu vamos usar o 'cur.execute' para ir buscar o id de todas as faturas associadas a esse doente nas consultas e nas cirurgias e vamos guardar isso tudo na variável 'id_faturas'. Depois vamos fazer um ciclo 'for' para iterar sobre cada fatura do doente atual e vamos usar o 'cur.execute' para ir buscar o valor pago de cada fatura e somar todos. De seguida vamos comparar esse preço total com os valores do dicionário e por na posição correspondente.

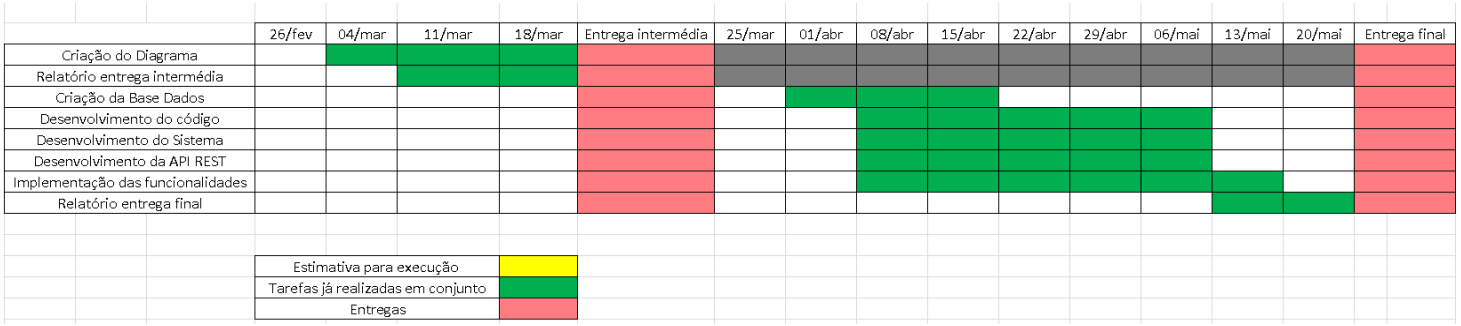
Por fim, vamos organizar os resultados e vamos selecionar o nome do utente, o preço pago, as informações de todas as consultas e cirurgias que contribuíram para o utente estar no top 3 e fechamos a conexão com a base de dados.

- **ENDPOINT REPORT**

Primeiramente vamos estabelecer conexão com o banco de dados e guardar o mês e o ano atual nas variáveis ‘mes_atual’ e ‘ano_atual’. Depois vamos iniciar um dicionário com 12 chaves que correspondem aos 12 meses. De seguida vamos buscar o id de todos os médicos e vamos iniciar um ciclo while que vai executar até o valor de ‘mes_atual’ ser maior que 0 em que vamos usar um ciclo for para iterar sobre todos os médicos e usando o ‘cur.execute’ vamos buscar o número de cirurgias de cada médico e verificar se para o mês atual este é o que fez mais cirurgias, a cada execução do ciclo while vamos adicionar +1 à ‘chave_atual’ e retirar 1 ao ‘mes_atual’. De seguida fazemos outro ciclo while que vai executar enquanto o valor da chave atual for menor que 12 e dentro desse ciclo voltamos a ir buscar o número das cirurgias de cada médico e verificar se ele é o que tem mais cirurgias neste mês, a cada execução do ciclo while vamos adicionar +1 ao valor da ‘chave_atual’ e retirar 1 ao valor de ‘mês_aux’.

Por fim, vamos organizar os resultados em que vamos imprimir o médico com mais cirurgias, e o número de cirurgias para cada mês nos últimos 12 meses e fechamos a conexão com a base de dados.

PLANO DE DESENVOLVIMENTO



	26/fev	04/mar	11/mar	18/mar	Entrega intermédia	25/mar	01/abr	08/abr	15/abr	22/abr	29/abr	06/mai	13/mai	20/mai	Entrega final
Criação do Diagrama															
Relatório entrega intermédia															
Criação da Base Dados															
Desenvolvimento do código															
Desenvolvimento do Sistema															
Desenvolvimento da API REST															
Implementação das funcionalidades															
Relatório entrega final															

Estimativa para execução	
Tarefas já realizadas em conjunto	
Entregas	

CONCLUSÃO

Em suma, o projeto proporcionou-nos uma compreensão mais aprofundada de como uma base de dados é utilizada, especialmente no contexto de um sistema de gestão hospitalar. Utilizando diversas ferramentas, conseguimos consolidar e reforçar os nossos conhecimentos, o que resultou em uma absorção mais profunda do conteúdo ensinado.